Final Coursework — Assessment 2

> Refer to the **Course Calendar** in `Module_Guide_UNAV.md` for the official release date, submission deadline, and public dissertation slot. Those dates are the single source of truth; timelines here are descriptive only.

Overview

The final coursework is an individual dossier plus a public defense worth **60%** of the module grade (30% written submission, 30% public dissertation). Choose **one** of the tasks below and produce a polished, reproducible analysis that demonstrates mastery of the omics/HPC toolchain developed during the module.

Common Requirements

- Submit a single PDF or HTML export (Markdown-first encouraged) through ADI.
- Include code listings, command logs, and links to scripts/notebooks stored in your repo.
- Embed 6–8 high-quality figures or tables with captions.
- Cite all external datasets, tools, and references (APA/Harvard acceptable).
- Document individual contributions when collaborating during ideation.
- Prepare a 10-15 minute PowerPoint presentation for the tribunal, covering Abstract → Introduction → Methods → Results → Conclusions.

Task 1 — Automated NGS Pipeline

Derive your solution from `CourseWork_Task1.Rmd` and extend it into a production-ready Bash workflow for the NGS1 practical (BQ.fq reads).

Minimum Checklist

1. **Script structure**: shebang, author/date/purpose block, usage instructions, parameter parsing (`$1…$4` → descriptive variable names).
2. **Pipeline stages**: QC (FastQC/MultiQC), optional trimming (Cutadapt), Bowtie2 alignment with parameter exploration, Samtools post-processing, MultiQC summary export.
3. **Automation**: accepts arbitrary FASTQ paths, echoes resolved commands before execution, logs stats to timestamped files, removes intermediates after summarising.
4. **Documentation**: inline comments outlining each stage, README snippet on how to run the script, and references to the Bash tutorials listed at the bottom of the original Rmd.

Deliverables

- Bash script(s) + helper configs.
- Sample run log demonstrating parameter tuning and its impact on mapping metrics.
- Short discussion on optimisation strategy and reproducibility safeguards.

Task 2 — Bacterial Genome Comparison

> Dear Universidad de Navarra students,
> We are terribly grateful for your advice on antibiotic prescriptions for our *E. coli* AFPN02.1 infected patients. After a few doses we have controlled the progression of the outbreak in most of the infected—your advice is saving human lives! Unfortunately, we still have extreme cases where the infection has systemically progressed and normal prescriptions no longer work. We need you to further identify the nature of the AFPN02.1 virulence-associated genes to treat our patients. Aside from antibiotic resistance genes, AFPN02.1 carries myriad defensive and offensive functions to infect, spread, and in some occasions kill the host. More virulence references are available in the course material folder; you may also incorporate any peer-reviewed sources to complement your investigation.
> Please keep pushing — people are counting on you.

Investigate virulence-associated genes in *E. coli* AFPN02.1 following the Session 3 comparative genomics workflow.

1. **Question 1 (pan-virulence panel)**
    - Build virulence gene sets from VFDB/PAIDB or the provided `/home/manager/Desktop/course_materials/genomes/wholeGenomeExamples/EHEC_vir.fn` file plus literature.
    - Annotate AFPN02.1 and comparator strains; identify presence/absence patterns via BRIG.
    - Recommend candidate targets for therapeutic intervention.
2. **Question 2 (deep dive)**
    - Select one virulence gene/operon present in AFPN02.1.
    - Describe its biological mechanism and run BLAST-derived phylogeny/orthology analysis.
    - More specifically, you should: • Find information on the biological action/mechanism of this gene • Study the phylogeny of the gene (again following the steps suggested in the practical)
3. **Question 3 (extension menu — choose any that fit your narrative)**
    1. Functional annotation of virulence-associated genes to pinpoint roles in pathogenesis.
    2. Pathway enrichment analysis to highlight biological modules or secretion systems involved.
    3. Structural assessments (domains/motifs) linking genotype to molecular function.
    4. Evolutionary analysis: phylogenies, gene duplication, selection pressures, horizontal transfer evidence.
    5. Regulatory network mapping (sigma factors, transcriptional regulators, non-coding RNAs).
    6. Comparative genomics across strains/species to identify conserved vs. lineage-specific determinants.
    7. Host-pathogen interaction profiling (e.g., host immune evasion markers, receptor binding predictions).
    8. Drug-target prediction or virtual screening concepts for prioritized virulence genes.

9. Epidemiological synthesis merging genomic findings with outbreak metadata.
10. Machine-learning or rule-based scoring of virulence genes to prioritise therapeutic targets.
11. Rapid response dashboard that visualises gene presence/absence, patient clusters, and recommended interventions for hospital staff.

Include code snippets, BRIG snapshots, and clearly articulated conclusions plus citations.

Task 3 — RNA-seq Differential Expression (DESeq2) **NOT AVAILABLE ON THE 2025-2026 COURSE**

Starting from the shared count table, complete the core workflow below and then explore any optional extensions that strengthen your biological narrative.

1. Filter DEGs on padj < 0.01 and |log2FC| > 2; print the top 10 genes (symbols, not Ensembl IDs).
2. Produce:
   - A volcano plot with gen symbol labels for the 20 most significant genes.
   - A heatmap covering the 30 most significant genes (symbols only).
3. Run GO over-representation tests for Molecular Function and Biological Process; report enriched terms.
4. Build Molecular Function and Biological Process concept networks / enrichment maps.
5. Identify KEGG pathways affected by dexamethasone treatment (e.g., using ClusterProfiler).
6. Compare differential expression across multiple experimental groups or contrasts (pairwise and group-wise).
7. Apply batch-effect mitigation (e.g., `removeBatchEffect` or DESeq2 design terms), then describe how the results change.
8. Deliver a structured report (introduction, computing code/methods, results, plots, citations).

Optional enhancements (tackle any that inspire you):

1. Functional annotation of the top gene clusters (e.g., Reactome, MSigDB Hallmarks).
2. Isoform-level or alternative splicing follow-ups (IsoformSwitchAnalyzeR).
3. Reproducibility automation (targets/Makefile/bash script) that rebuilds all figures.
4. Benchmark your DESeq2 results against another method (edgeR, limma-voom) and discuss discrepancies.
5. Build an interactive dashboard (Shiny, Quarto) summarising DE findings for clinicians.

Task 4 — Biomedical Data Exploration and Statistical Analysis

The task in Assesment_R_Programming_Basics.Rmd is to analyze biomed_data.csv in R and write a short report. Students need to fit a linear model and a two-way ANOVA, handle missing data, create a few plots and grouped summaries, and then choose several extra exploratory questions for a deeper analysis. The final deliverable is a knitted R Markdown report with code, results, and brief interpretations.

Develop your solution from the Assessment_R_Programming_Basics material and turn it into a clear, reproducible R Markdown report based on biomed_data.csv. The goal is to explore the cohort, test a few simple statistical models, and communicate the results with plots, tables, and short interpretations.

Minimum Checklist

1. Environment setup and data import: load the required R packages, import biomed_data.csv, and inspect the structure, summaries, and missing values.
2. Linear modelling: fit a multiple regression model for expression using marker_B, dose, and group; report the main coefficients and interpret them in context.
3. Model diagnostics: produce the standard diagnostic plots, identify possible outliers or non-constant variance, and comment on what should be checked next.
4. Two-way ANOVA: test the group × dose interaction on expression and explain whether treatment effects depend on dose.
5. Missing-value handling: impute marker_C with the median, compare the distribution before and after imputation, and describe the impact.
6. Grouped visualisation: create age_group categories, plot heart_rate by age group with jittered points, and discuss visible trends.
7. Data reshaping and faceting: pivot the marker variables longer, draw violin plots by marker type and group, and identify markers with the clearest group differences.
8. Extended analysis: choose at least seven additional exploratory questions from the prompt list, such as correlations, PCA, clustering, outlier detection, or regional comparisons.

Deliverables

- A completed R Markdown report with code, tables, plots, and short written interpretations.
- A knitted HTML or PDF version of the report.
- Any extra exported tables or figures used in the analysis if relevant could be zipped.

Task 5 — Optional TFM Fast-Track

Pitch the opening package of your Máster Final Project, ensuring tight alignment with the module’s omics/HPC scope.

1. **Advisor agreement**: meet with Igor to define topic boundaries, deliverables, and evaluation criteria; include a signed or emailed summary.
2. **Concept note**: 2–3 page narrative tying objectives, datasets, and computational methods to module competencies and the broader TFM roadmap.
3. **Pilot evidence**: prototype scripts, preliminary analyses, or workflow diagrams demonstrating feasibility within the module timeline.
4. **Reflection**: describe how completing this fast-track reduces risk for the full TFM.

Evaluation Snapshot

Component	Weight	Notes

Component	Weight	Notes
NGS Hackathon (Assessment 1)	40%	See module guide.
Final Coursework PDF/HTML	30%	Rubric emphasises scientific depth, clarity, and reproducibility.
Public Dissertation	30%	Presentation, defense quality, and ability to answer tribunal questions.

A minimum mark of **5/10** is required in each component. Plan backwards from the Course Calendar milestones and keep your advisor informed of progress.